MINI PROJET ALGORITHMIQUE

La Bataille Navale en PHP

Compte Rendu de Projet

Réalisé par :

Grauet Mathis & Andries Alexis

Filière : Informatique — Algorithmique

Année : 2024 – 2025

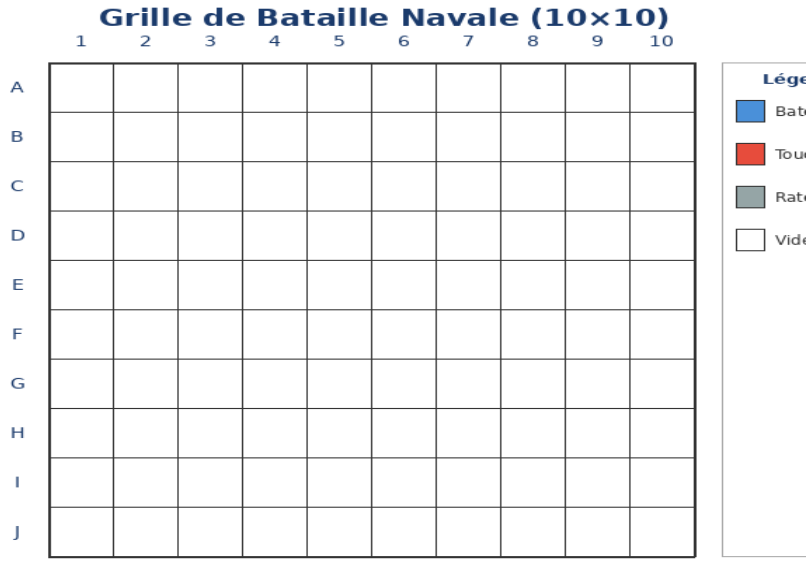


Figure 1 — La grille 10×10 de la Bataille Navale

1. Introduction

Ce document constitue le compte rendu du mini-projet d'algorithmique réalisé dans le cadre du cours d'informatique. L'objectif était d'implémenter en langage PHP le jeu classique de la Bataille Navale, en s'appuyant sur les notions fondamentales d'algorithmique : tableaux 2D, fonctions, boucles, gestion des entrées/sorties et logique conditionnelle.

Le jeu développé supporte deux modes de jeu : un mode solo (joueur contre l'ordinateur) et un mode deux joueurs (tour par tour sur la même machine). Le code est entièrement exécutable en ligne de commande (CLI) via l'interpréteur PHP.

1.1 Objectifs du projet

- Implémenter les règles complètes du jeu de la Bataille Navale
- Développer une architecture modulaire basée sur des fonctions PHP réutilisables
- Gérer les deux modes de jeu : solo et multijoueur
- Assurer la robustesse du programme face aux saisies invalides
- Appliquer les concepts vus en cours (tableaux, boucles, fonctions, récursivité)

2. Règles du Jeu

La Bataille Navale est un jeu de stratégie et de déduction pour 2 joueurs. Chaque joueur dispose d'une grille de 10×10 cases (colonnes 1–10, lignes A–J) sur laquelle il positionne sa flotte de navires, de manière horizontale ou verticale. Les grilles sont cachées à l'adversaire.

2.1 La Flotte

Chaque joueur doit positionner les navires suivants sur sa grille avant le début de la partie :

Navire	Taille	Quantité	Variable PHP
Porte-avions	5 cases	1	porte_avions
Croiseur	4 cases	1	cuirasse
Sous-marin	3 cases	2	sous_marin
Torpilleur	2 cases	1	destroyer

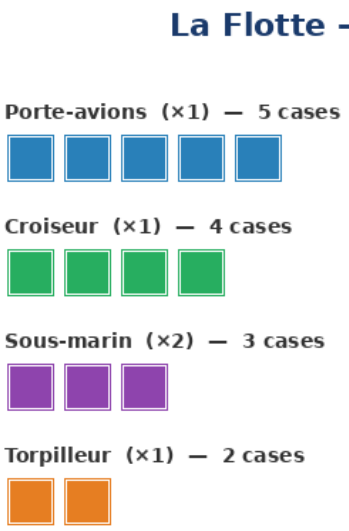


Figure 2 — Représentation visuelle des navires de la flotte

2.2 Déroulement

À tour de rôle, chaque joueur désigne une case ennemie par ses coordonnées (ligne, colonne). Si la case contient un navire, c'est un « Touché » (marqué par une croix). Sinon, c'est un « Raté » (marqué par un rond). Le joueur qui touche ou coule un navire rejoue. La victoire appartient au premier joueur qui coule l'intégralité de la flotte adverse.

3. Architecture du Programme

Le programme PHP est structuré de manière modulaire : chaque fonctionnalité du jeu est encapsulée dans une fonction dédiée. Le programme principal (en bas du fichier) sert uniquement de point d'entrée et d'aiguillage vers les modes de jeu.

3.1 Structures de Données

La grille de jeu est représentée sous forme de tableau PHP bidimensionnel (tableau de tableaux). Chaque cellule contient un entier dont la valeur encode l'état de la case :

- 0 — Case vide (aucun tir effectué, pas de bateau)
- 1 — Bateau intact (case occupée, non encore ciblée)
- 2 — Touché (bateau présent et ciblé avec succès)
- 3 — Raté (tir à l'eau, aucun bateau sur cette case)

```
// Initialisation d'une grille 10×10
$grille = initialisegrille(10, 10);  // Remplie de 0

// Tableau des types de bateaux
$bateaux = [
    "porte_avions" => ["taille" => 5, "nb" => 1],
    "croiseur"     => ["taille" => 4, "nb" => 1],
    "sous_marin"   => ["taille" => 3, "nb" => 2],
    "destroyer"    => ["taille" => 2, "nb" => 1]
];
```

Une seconde structure $bateauxInfo, de même dimensions que la grille, mémorise le nom du navire occupant chaque case. Cela permet à la fonction tire() d'identifier quel bateau vient d'être touché et de décrémenter ses points de vie (PV). Enfin, un tableau associatif $pvBateaux associe chaque navire à ses PV courants.

3.2 Vue d'Ensemble des Fonctions

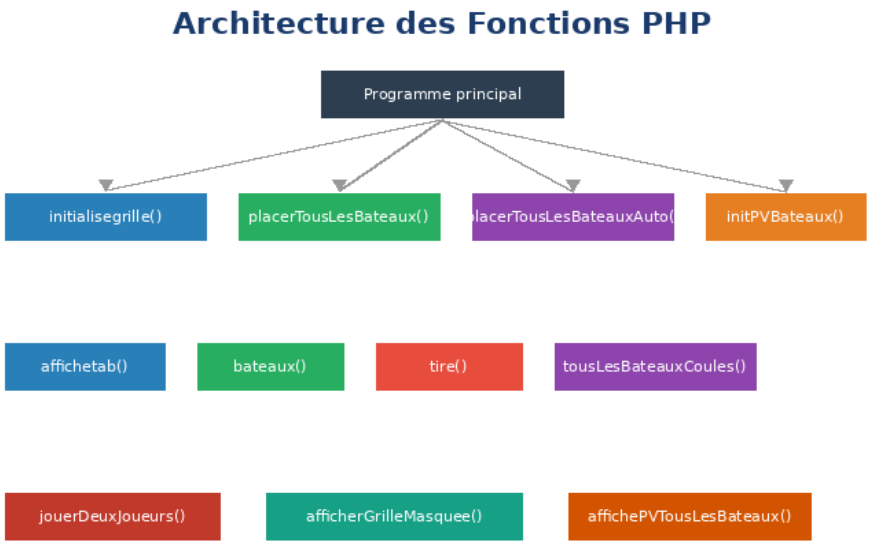


Figure 3 — Diagramme d'architecture des fonctions PHP du projet

4. Détail des Fonctions

Le tableau ci-dessous recense l'ensemble des fonctions implémentées dans le fichier code.php, avec leur rôle respectif :

Fonction	Rôle
`initialisegrille()`	Crée une grille 2D (tableau PHP) remplie de 0
`bateaux()`	Place un bateau sur la grille avec validation
`placerTousLesBateaux()`	Placement manuel interactif de tous les navires
`placerTousLesBateauxAuto()`	Placement aléatoire automatique des navires
`tire()`	Gère un tir : touché (2), raté (3), déjà tiré
`initPVBateaux()`	Initialise les points de vie de chaque bateau
`tousLesBateauxCoules()`	Vérifie si tous les PV sont à 0 (condition victoire)
`afficherGrilleMasquee()`	Affiche uniquement les tirs (masque les bateaux intacts)
`affichePVTousLesBateaux()`	Affiche l'état de chaque navire avec ses PV restants
`jouerDeuxJoueurs()`	Orchestre la partie en mode 2 joueurs tour par tour
`clearScreen()`	Efface la console (séquences ANSI) entre les tours
`affichetab()`	Affiche le tableau 2D avec en-têtes ligne/colonne

4.1 Fonction bateaux() — Placement avec Validation

La fonction bateaux() est la pierre angulaire du placement. Elle accepte la grille, les coordonnées de départ, le sens ("verticale" ou "horizontal"), la taille du navire et son nom. Avant de placer le bateau, elle effectue plusieurs contrôles de validité :

- Vérification que le sens fourni est bien 'verticale' ou 'horizontal'
- Contrôle de débordement : le bateau ne doit pas sortir de la grille
- Contrôle de chevauchement : toutes les cases ciblées doivent être à 0

Si toutes les vérifications passent, les cases sont marquées à 1 et les noms de bateau enregistrés dans bateauxInfo. La fonction retourne true en cas de succès, false sinon.

4.2 Fonction tire() — Gestion des Tirs

La fonction tire() reçoit les coordonnées du tir, la grille cible, et optionnellement les structures de PV. Elle gère les quatre cas possibles : case vide (→ 3, raté), case déjà ciblée (→ message d'erreur, return false), bateau intact (→ 2, touché, décrémentation des PV). Si les PV tombent à 0, le message « COULÉ » est affiché.

```
// Extrait de tire()
if ($grille[$ligne][$colonne] == 0) {
    $grille[$ligne][$colonne] = 3;  // Raté
    echo "Coup dans l'eau !\n";
    return true;
}
if ($grille[$ligne][$colonne] == 1) {
    $grille[$ligne][$colonne] = 2;  // Touché
    $pvBateaux[$nomBateau]--;
    if ($pvBateaux[$nomBateau] <= 0) echo "COULÉ !\n";
    return true;
}
```

5. Modes de Jeu

5.1 Mode Solo — Joueur contre Ordinateur

En mode solo, deux grilles sont initialisées : une pour l'ordinateur (bateaux placés automatiquement et cachés) et une pour le joueur. Le joueur choisit son mode de placement (automatique ou manuel). La boucle de jeu alterne entre le tir du joueur sur la grille de l'ordinateur et le tir aléatoire de l'ordinateur sur la grille du joueur.

L'ordinateur tire aléatoirement en évitant les cases déjà ciblées. Un compteur de tirs permet d'afficher le score final. La partie s'arrête dès qu'un des deux belligérants n'a plus aucun navire à flot.

> Fonctionnalité notable : le joueur peut choisir le niveau de détail de l'affichage.
> Mode détaillé : les noms des navires et leurs PV restants sont affichés en temps réel.
> Mode simplifié : seule la notification de coulage est affichée, sans révéler les noms.

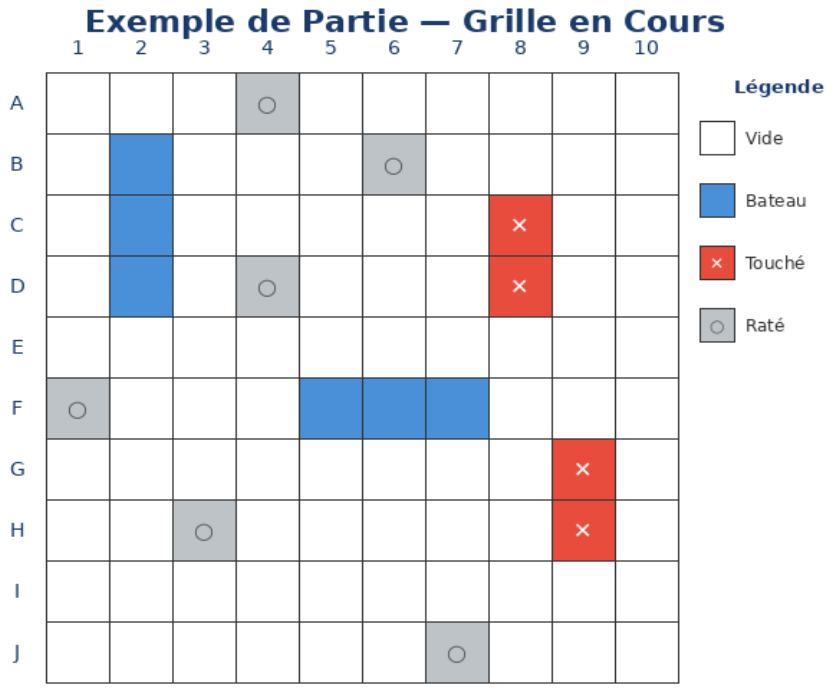


Figure 4 — Exemple de grille en cours de partie (rouge=touché, gris=raté)

5.2 Mode Deux Joueurs — Tour par Tour

Le mode deux joueurs gère deux joueurs humains sur la même machine. Pour préserver le secret des positions, la fonction clearScreen() efface le terminal entre chaque tour à l'aide de séquences d'échappement ANSI. Chaque joueur place ses bateaux (mode auto ou manuel) avant le début de la partie.

La boucle de jeu alterne les tours : le joueur actif saisit ses coordonnées, le résultat est affiché, puis la grille adverse masquée (ne montrant que les tirs effectués) est affichée. Le joueur doit appuyer sur Entrée avant de passer la main pour éviter que l'adversaire voie l'état du jeu.

6. Déroulement d'un Tour et Analyse

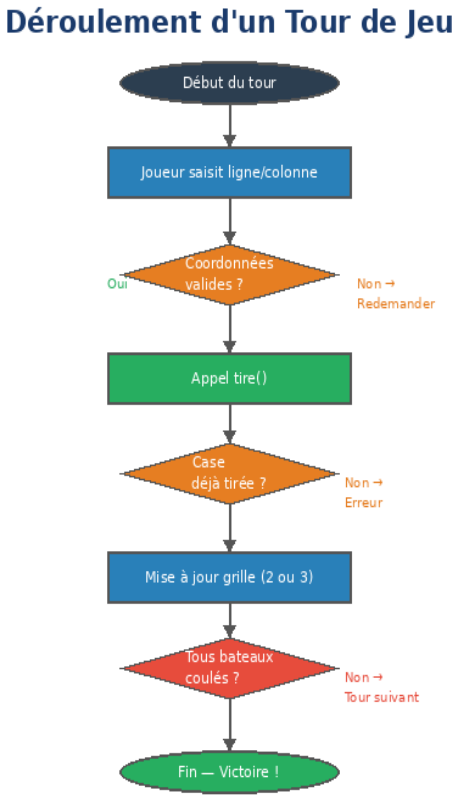


Figure 5 — Flowchart du déroulement complet d'un tour de jeu

6.1 Gestion des Erreurs

Une attention particulière a été portée à la robustesse du programme face aux entrées invalides. Chaque saisie utilisateur fait l'objet de contrôles explicites :

- Coordonnées hors grille (< 0 ou > 9) : message d'erreur et redemande
- Case déjà ciblée : tire() retourne false, la boucle force une nouvelle saisie
- Placement hors grille ou chevauchement : bateaux() retourne false avec message
- Mode de jeu invalide : arrêt propre du programme avec exit

6.2 Points Forts du Projet

- Modularité : chaque fonctionnalité est isolée dans sa propre fonction
- Réutilisabilité : les fonctions tire() et bateaux() sont partagées entre les deux modes
- Placement aléatoire efficace avec mécanisme de tentatives multiples (max 5000)
- Double mode de jeu (solo / multijoueur) dans un seul fichier PHP
- Gestion des PV par bateau permettant de détecter les coulages précis

6.3 Améliorations Possibles

Bien que fonctionnel, le projet pourrait être enrichi de plusieurs façons :

- Intelligence artificielle améliorée : l'ordinateur cible actuellement de manière purement aléatoire. Une IA stratégique (recherche par quadrant après un touché) améliorerait le défi.
- Interface graphique : migration vers une interface web avec HTML/CSS/JavaScript ou une interface ncurses en CLI.

- Sauvegarde de partie : sérialisation de l'état du jeu en JSON pour permettre de reprendre une partie interrompue.
- Mode réseau : utilisation des sockets PHP pour jouer sur deux machines distinctes, évitant le partage d'écran.

6.4 Conclusion

Ce mini-projet d'algorithmique nous a permis de mettre en pratique l'ensemble des notions fondamentales du cours dans un contexte ludique et concret. L'implémentation de la Bataille Navale en PHP a nécessité la conception d'une architecture claire, une gestion rigoureuse des structures de données à deux dimensions, et une attention constante à la robustesse face aux interactions utilisateur.

Le projet remplit tous les objectifs fixés : les deux modes de jeu (solo et multijoueur) sont pleinement fonctionnels, la flotte complète est gérée avec suivi des PV, et les règles officielles du jeu sont respectées. Ce travail constitue une base solide qui pourrait être étendue avec les pistes d'amélioration identifiées ci-dessus.